PRML Lab 3 — Bagging & Boosting

Aryan Tiwari (B20AI056)
CSL2050 Pattern Recognition & Machine Learning, Winter 2022

Implementation note. For Q1 tasks 1–8 and Q2 tasks 1–3 I wrote a CART decision tree from scratch. It handles both regression (the split minimises the children's summed squared error; leaves predict the mean) and classification (the split minimises weighted Gini impurity; leaves predict the majority class). At each node every feature is sorted once and all thresholds are scored in one pass with cumulative sums. Random Forest, AdaBoost, XGBoost and LightGBM come from their libraries, as the assignment allows.

Question 1 — Housing prices (regression)

Pre-processing. 545 houses, 12 features, no missing values or duplicates. The six yes/no columns were mapped to 0/1 and `furnishingstatus` was encoded as an ordinal variable (unfurnished < semi < furnished). Price was converted to lakhs so that errors are easy to read. Feature scaling is unnecessary because tree splits are unaffected by monotonic transformations. 80:20 split: `Xtrain` = 436 rows, test = 109 rows.



Figure 1: Price is right-skewed; area, bathrooms and air-conditioning correlate most with price.

1. Single tree without validation. A fully-grown tree reaches $R^2 = 0.999$ on the training data but only **0.421** on the test set. It memorises the training data (the train R^2 falls just short of 1 because a few houses have identical features but different prices) and generalises poorly, which is the high-variance problem that motivates bagging.

2–3. 5-fold CV for `max_depth`. Mean validation R^2 peaks at **depth 3 (0.461 ± 0.120)**. Shallower trees underfit. Deeper trees overfit: by depth 10 the mean drops to 0.25. The spread between folds is large (fold 3 is the easiest at every depth, while folds 1 and 2 are the hardest), so a single train/validation split would have given an unreliable choice of depth.

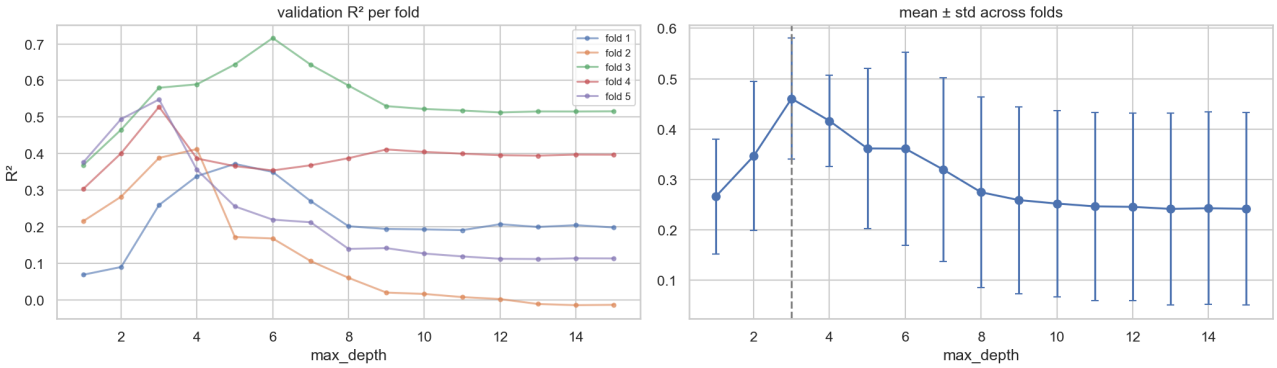


Figure 2: Validation R^2 for each fold (left) and mean $\pm$ std (right).

4–7. Bagging. I drew ten bootstrap samples (each the size of `Xtrain`, drawn with replacement, so about 63% of rows are unique) and trained one depth-3 tree on each. On the test set the individual trees score R^2 between 0.349 and 0.481, averaging $\mathbf{0.425 \pm 0.041}$. Averaging their predictions gives $\mathbf{R^2 = 0.524}$ — better than every single tree. Averaging cancels the parts of each tree's errors that are not shared with the others, so variance falls while bias stays the same.

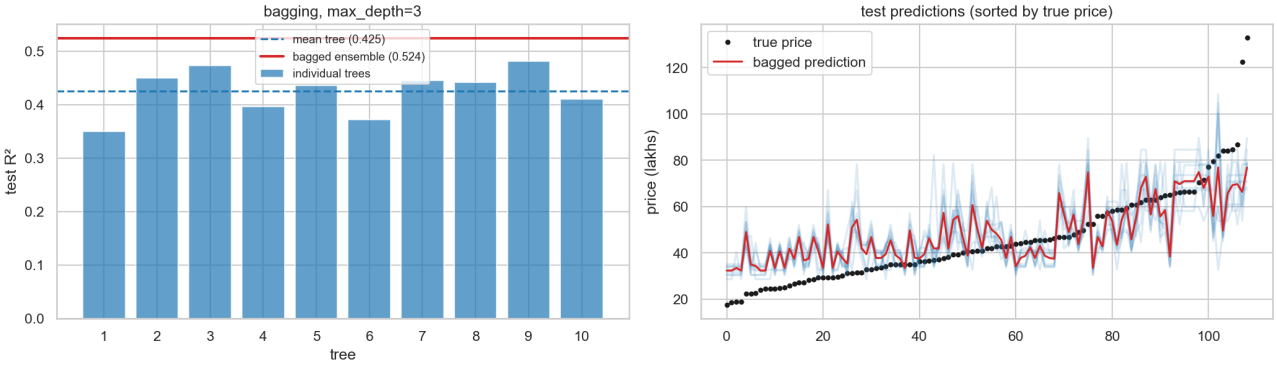


Figure 3: Per-tree vs. bagged R^2 (left); the bagged prediction (red) is a smoothed version of the 10 trees (blue).

8. Changing `max_depth`. The gain from bagging grows with depth: +0.03 R^2 at depth 1, +0.10 at depth 3, +0.25 at depth 8 and +0.28 for unlimited depth. Deep trees have low bias and high variance, which is exactly what averaging removes, so the bagged R^2 *rises* to about 0.62 even though each deep tree on its own gets *worse* (0.34). Shallow trees have high bias and low variance, and averaging cannot fix bias. So when bagging, you should grow deep trees, not the depth that is optimal for a single tree.

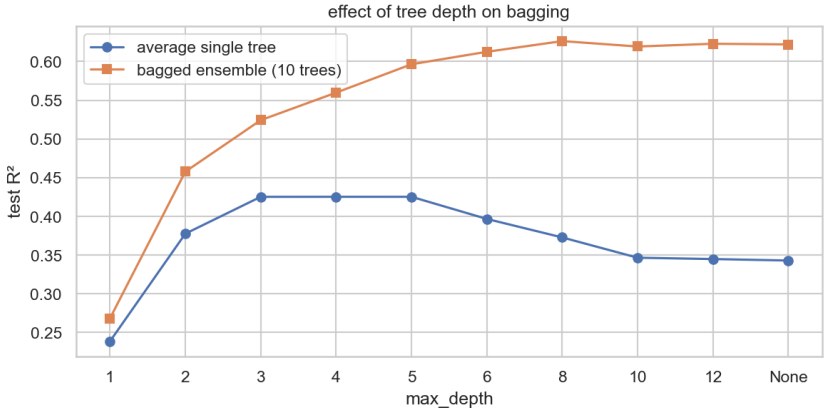


Figure 4: Average single-tree vs. bagged test R^2 as tree depth increases.

9–10. Random Forest and AdaBoost.

Model (test set)	MSE (lakh^2)	MAE (lakh)	R^2
Single tree, depth 3	211.1	11.00	0.480
Bagging, 10 trees (scratch)	193.1	10.50	0.524
Random Forest, 200 trees	**140.2**	**8.37**	**0.655**
AdaBoost, 200 estimators	178.5	10.20	0.560

Random Forest is clearly best. It uses deep trees, many more of them, and random feature subsets at each split, which makes the trees less correlated, so averaging reduces variance more than plain bagging does. AdaBoost improves on a single tree by focusing successive learners on the houses with the largest errors, but price has heavy-tailed noise and that limits it.

Question 2 — Breast cancer (classification)

569 samples, 5 numeric features, labels 357 / 212, no missing values. Radius, perimeter and area are almost perfectly correlated (all three measure size). 80:20 stratified split (455 / 114).

1. Single fully-grown tree. Train accuracy 100%, test accuracy **91.2%**.

2–3. 5-fold CV for `max_depth`. Mean validation accuracy is highest at **depth 5 (89.9% ± 1.6%)**; refitting at that depth raises test accuracy to **94.7%**, compared with 91.2% for the unpruned tree. The CV curve is fairly flat (87–90%), because a single split on size already separates most cases.

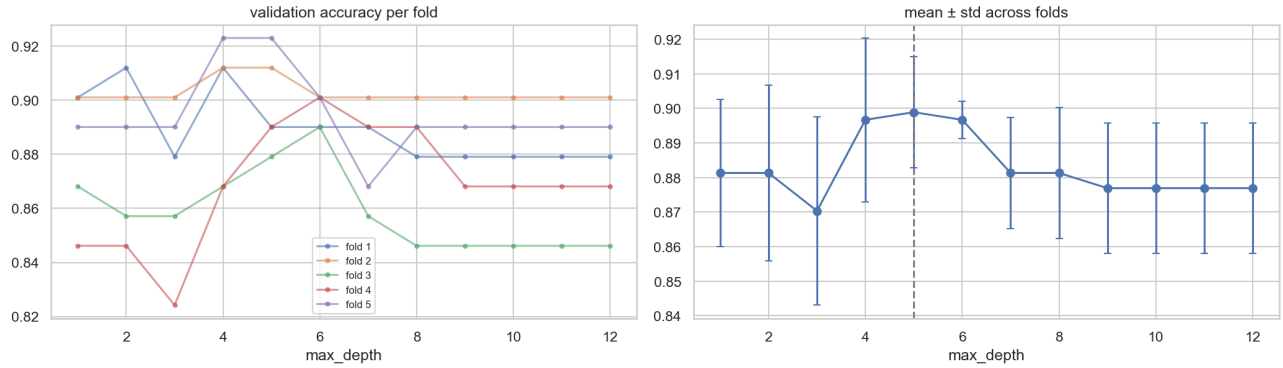


Figure 5: Validation accuracy per fold and mean ± std for the scratch classifier.

4–5. XGBoost (`subsample`=0.7, `max_depth`=4, 200 rounds): train accuracy 100%, test accuracy **95.6%**. Row subsampling makes each boosting round see a different 70% of the data, which acts like bagging inside boosting.

6–7. LightGBM, `max_depth` vs. `num_leaves`. A tree of depth d has at most 2^d leaves, so with `max_depth`=3 any `num_leaves` > 8 has no effect: every metric is identical from 8 leaves up. LightGBM grows trees *leaf-wise*, so `num_leaves` is its main complexity control and `max_depth` is a cap on top of it. On a 114-sample test set accuracy only moves in steps of 0.9%, so I also tracked log-loss:

`num_leaves`	2	3	4	6	8	16
test log-loss, depth 3	0.120	**0.106**	0.107	0.114	0.144	0.144
test log-loss, no depth limit	0.120	**0.106**	0.107	0.117	0.144	0.248
train log-loss, no depth limit	0.121	0.062	0.031	0.009	0.002	0.000

Test loss is lowest at 3–4 leaves. **Overfitting starts at about 6 leaves**: train loss keeps falling towards zero while test loss rises. Without a depth cap it more than doubles by 16 leaves (0.248). In

accuracy terms, train accuracy reaches 100% as soon as `num_leaves` ≥ 4 (with depth ≥ 3) while test accuracy stays at about 95–96%.

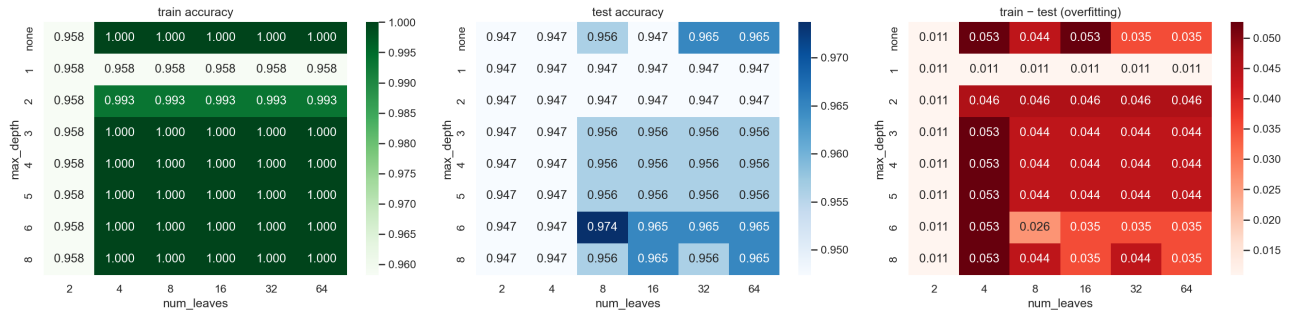


Figure 6: Train accuracy, test accuracy and their gap over `max_depth` × `num_leaves` (none = no limit).

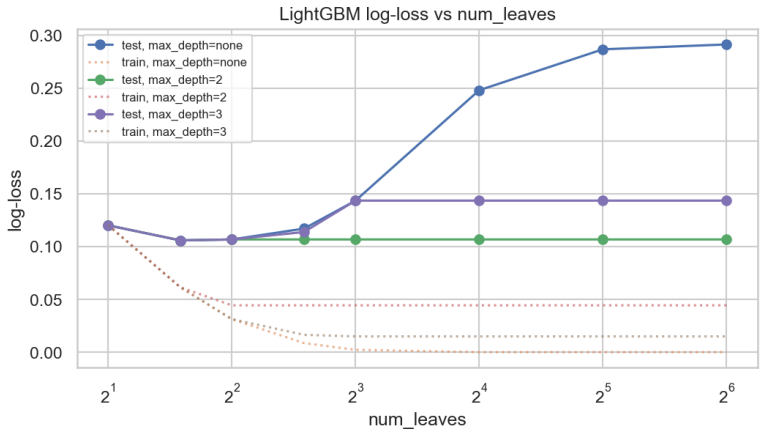


Figure 7: Log-loss vs. `num_leaves`; dotted lines are training loss.

8. Parameters for accuracy vs. overfitting.

- *More capacity / accuracy:* `num_leaves`, `max_depth`, `n_estimators` together with a smaller `learning_rate`. These help until the model starts fitting noise.
- *Controlling overfitting:* keep `num_leaves` below $2^{\texttt{max_depth}}$; raise `min_child_samples` (minimum data in a leaf); use row and feature subsampling (`bagging_fraction`/`subsample`, `feature_fraction`/`colsam` use L1/L2 regularisation (`lambda_l1`, `lambda_l2`); and use early stopping on a validation set.

On this dataset a very small model (3–4 leaves) generalises best, because the classes are almost separable by tumour size.